

A Masterclass for All Important Generative AI Terminologies

Ideal for Interviews — A Complete Step-by-Step Tutorial Designed for Absolute Beginners

1. Introduction: Why Generative AI is a Paradigm Shift

For decades, computers operated under a strict rules-based system. If you wanted a computer to recognize a cat, a programmer had to manually write rules defining cat ears, whiskers, and shapes. This is known as **Traditional Programming**.

Generative Artificial Intelligence (Generative AI) turns this upside down. Instead of writing rules, we feed a massive computer system billions of examples of human writing, imagery, or music. The system learns the underlying hidden patterns on its own. Once trained, it doesn't just categorize data—it can *generate* entirely brand-new content that has never existed before.

2. Core Language Engine Building Blocks

Before jumping into how artificial brains think, let's look at the basic knobs, switches, and data packages that control large AI programs called **Large Language Models (LLMs)**.

Tokens: How Models Read Human Text

Computers cannot read characters or words the way humans do because computers only understand math. When you type text into an AI, it runs through a component called a **Tokenizer**. This tool slices words up into small, atomic text puzzle pieces called **Tokens**.

Step-by-Step Mechanical Example:

Consider the word: "unbelievable".

A tokenizer doesn't see one word. It slices it into fragment chunks based on patterns it knows: ["un", "believ", "able"].

Each unique chunk in the AI's memory has a specific ID number assigned to it. For example: "un" → 412, "believ" → 1823, "able" → 94. This array of numbers is what the AI actually processes.

Why this matters: Every model has a physical storage limit called a **Context Window**. This is the maximum number of tokens the model can hold in its temporary memory space during a single chat conversation. If you exceed this limit, the model will cleanly forget the beginning of your conversation.

Parameters: The Internal Brain Weights

When people say a model has 8 Billion or 70 Billion parameters, think of parameters as tiny adjustable volume knobs inside a vast neural network calculation board. Each parameter is a decimal number (like `0.1432` or `-0.9811`) that controls how strongly a signal passes from one simulated brain cell (neuron) to another.

The Training Analogy:

When an LLM is created, all its billions of knobs are randomized. It treats data like complete gibberish.

During

Training

, the model reads billions of pages of text. If it guesses the next word wrong, a mathematical optimization formula slightly tweaks these parameter knobs. Over months of massive scale computation, these knobs align so that the system accurately mirrors human thought and grammar logic.

Temperature: Controlling Randomness

When an LLM responds, it calculates a list of options for the very next token alongside a probability percentage. Temperature is a math multiplier that adjusts these percentages before the final word is picked.

- **Low Temperature (0.0):** The AI will exclusively pick the number-one most probable word every single time. It becomes perfectly consistent and literal. *Best for: Math and writing computer code.*
- **High Temperature (0.9+):** The probability differences between the top words are flattened out. The AI is allowed to pick less expected, erratic tokens. *Best for: Creative fiction or brainstorming.*

3. The Ultimate Structural Architecture: The Transformer Model

Before 2017, language software processed words one by one in order, like a train car pulling out of a station. If a system read a long paragraph, it would often completely forget the first word by the time it reached the last word. In 2017, researchers introduced the **Transformer Architecture**, which allows a computer to look at *every word in a document simultaneously*.

Step-by-Step Process of a Transformer Functioning

1. **Embedding:** Words are turned into numbers, and then those numbers are translated into an array of coordinates in space called an **Embedding Vector**. Words with close definitions are automatically placed close together in this map.
2. **Positional Encoding:** Because the model reads all words at once, it loses track of word order. The system injects a specialized wave calculation value into each word vector to tag its exact place in the sentence structure.

3. **The Self-Attention Mechanism:** This is the core engine. For every single token, the model calculates three intermediate vector states using its internal parameter matrices:
- **Queries (Q):** What the current word is looking for.
 - **Keys (K):** What identity value the other words in the text contain.
 - **Values (V):** The actual meaning or details of the word.

The mathematical equation that runs this attention layer is written as:

$$\text{Attention}(Q, K, V) = \text{Softmax}((Q \times K^T) / \sqrt{d_k}) \times V$$

Ground-Up Breakdown of the Math:

- **$Q \times K^T$ (The Comparison):**

The model takes the Query of one word and multiplies it by the Keys of all other words. This creates a matrix of scores showing how relevant every word is to every other word. For example, in the sentence *"The bank of the river"*, the word *"bank"* will have an incredibly high relationship score with *"river"*, helping the model realize this is land, not a financial institution.

- **$\sqrt{d_k}$ (The Scale Regulator):**

This divides the raw scores by a scaling factor based on the model's structural size. It prevents numbers from becoming too massive, keeping calculations stable.

- **Softmax (The Normalizer):**

This turns the raw relation scores into simple percentages that add up to exactly 100%.

- **Multiplying by V (The Output Weight):**

This percentage map is multiplied by the Value vectors, filtering out useless words and outputting a highly accurate contextual representation of the text.

Modern Enhancements to the Transformer

- **Mixture of Experts (MoE):** Instead of activating all 100+ billion parameter knobs for every single word, the network splits its internal layers into smaller specialist blocks called "Experts." A central "Router" directs each word only to the specific expert blocks it needs (e.g., a math expert block or a coding expert block), cutting hardware costs and speeds drastically.
- **Grouped-Query Attention (GQA):** Early transformers required massive amounts of memory to cache historical conversational tokens. GQA fixes this by forcing multiple Query tracking lines to share a single Key and Value stream, dramatically reducing memory usage.

4. Orchestration Frameworks: LangChain & LangGraph

Once you have access to an LLM, building an advanced enterprise software app around it manually becomes incredibly difficult. Orchestration frameworks act as the management scaffolding built around an LLM.

What Happens Without a Framework?

If you use raw code, you have to write manual, error-prone lines to format text inputs, handle API drops, and pass text from one calculation step to another inside basic loops. If the model fails or returns slightly unexpected text, the entire application crashes instantly.

LangChain: Linear Pipeline Orchestration

LangChain provides prefabricated tools to tie text parts together seamlessly into a straight, forward-moving pipeline called a **Chain**. It uses **LangChain Expression Language (LCEL)** to let developers string components together using a simple pipe symbol (`|`).

Linear Pipeline Model Flow:

```
User Raw Question → [Prompt Formatting Template] → [LLM Prediction Engine] → [Output Text Cleaner] → Final Answer
```

LangGraph: Cyclical Loops & Independent Agents

When an application needs to evaluate its own work, double-check code, or loop back to fix its mistakes, LangChain's straight-line structure breaks down. LangGraph treats a workflow like a map or **Graph** made of **Nodes** (actions) and **Edges** (conditional direction choices).

Cyclical Workflow Example:

An AI agent is tasked with writing code. Node A drafts the code. It passes the code to Node B, which attempts to run it. If Node B detects an error, an edge routes the traceback message *backward* to Node A. Node A reads the error, updates its code, and passes it forward again. This continuous loop runs until the code works perfectly.

5. Production RAG (Retrieval-Augmented Generation)

An LLM is frozen in time once its training finishes; it does not know about your private company files, latest news, or custom updates. **Retrieval-Augmented Generation (RAG)** gives the model an "open-book exam" capability by finding relevant information and inserting it into the prompt context on the fly.

The 4-Step RAG Architecture Pipeline:

1. Chunking (Segmentation):

Long text files are chopped into smaller, cohesive text pieces called chunks. Rather than slicing blindly by character count, *Recursive Character Chunking* splits text gracefully at paragraphs or sentence boundaries so that structural meanings stay completely intact.

2. Embedding (Mathematical Translation):

Each chunk runs through an embedding neural network that converts human text into a list of coordinates called a vector. Similar concepts are mapped near each other in this hidden vector coordinate space.

3. Indexing (Vector Database Storage):

These coordinate vectors are stored inside specialized **Vector Stores**

(like Pinecone or Qdrant). They act like highly specialized index catalogs for rapid geographic calculation lookups.

4. Retrieval & Generation:

When a user asks a question, the application turns that question into a vector coordinate, searches the database for the top matching reference text chunks, pastes those chunks into the prompt window context, and hands it to the LLM to write a well-informed response.

6. Comparative Matrix: Frameworks & Models

Framework Comparison

Framework	Core Structural Paradigm	Ideal Project Fitment
LangChain	Linear Pipelines (DAGs) without loops. Straightforward input-to-output chains.	Basic RAG searching, document summarizing pipelines, data extraction utilities.
LangGraph	Cyclical, state-persistent workflows with loops and conditional routing.	Complex autonomous agents, self-correcting coding bots, interactive multi-step systems.

Large Language Model Matrix (Tokens, Parameters, and Context Windows)

Model Family & Provider	Standard Parameter Scale	Maximum Context Window Limit	Differentiating Structural Superpower
OpenAI GPT-4o / o1 / o3	Proprietary Dense & MoE Mixed Scales	128,000 Tokens	Gold standard for robust instruction-following, balanced latency, and raw complex reasoning depth.
Anthropic Claude 3.5 / 4.7	Proprietary High-Capacity Scales	200,000 Tokens	Unmatched stability in executing logical trees and generating structured computer code safely.
Google Gemini 2.5 / 3 Pro	Proprietary Multimodal Native Scaling	2,000,000 Tokens	Native multimodal ingestion with vast multi-million token context windows to read massive repositories instantly.
Meta Llama 3.3 / Llama 4	8 Billion to 405 Billion (Open-Weights)	128,000 Tokens	Open-weights delivery allowing direct modification and full on-premise cloud installation without leaking data.
DeepSeek-R1 / V3	671 Billion Total (37 Billion Active MoE)	128,000 Tokens	Advanced reinforcement learning reasoning that explicitly renders its thinking process out loud using chain-of-thought tokens.